

App Development Project Report

1. Student Details

Name: Aman Sahu

Roll Number: 24f1000492

Email: 24f1000492@ds.study.iitm.ac.in

About Me: An aspiring Data Scientist and Software Developer pursuing a BS in Data Science and Applications at IIT Madras. Specialized in architecting scalable web applications and data-driven systems, focusing on bridging the gap between complex backend logic and user-centric frontend experiences.

2. Project Details

Project Title: Peak Mist – Trekking Management App

Problem Statement:

To design and build a web-based trekking management platform that allows administrators, trek staff (guides), and trekkers (users) to manage the full lifecycle of a trek — from creation and staff assignment, to discovery and booking by users, to on-ground participant management — all through role-specific dashboards.

Approach:

The app was built using Flask as the backend framework with a modular, blueprint-based structure separating Auth, Admin, Staff, and User concerns. Server-rendered Jinja2 templates provide role-specific dashboards, while SQLAlchemy models capture users, staff profiles, treks, and bookings. Flask-Login handles authentication and session management, with a role-based access decorator restricting each blueprint's routes to the appropriate user type. A self-registration and admin-approval workflow governs how new trek staff accounts are activated.

3. AI/LLM Declaration

I used chatgpt 5.5 to understand the models and there relationship like one to many and one to one.

I also used it for understand the ID logic of users(U001), Treks(T001) and Staff (S001)

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask Blueprints	Modular routing, separated by role (auth, admin, staff, user)
SQLAlchemy (Flask-SQLAlchemy)	Object Relational Mapper for the SQLite database
Flask-Login	User authentication and session management
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap	Frontend styling and responsive layout
Werkzeug (security)	Password hashing for secure credential storage
SQLite	Lightweight local relational database
Pytest	Automated tests (e.g. display-ID generation logic)

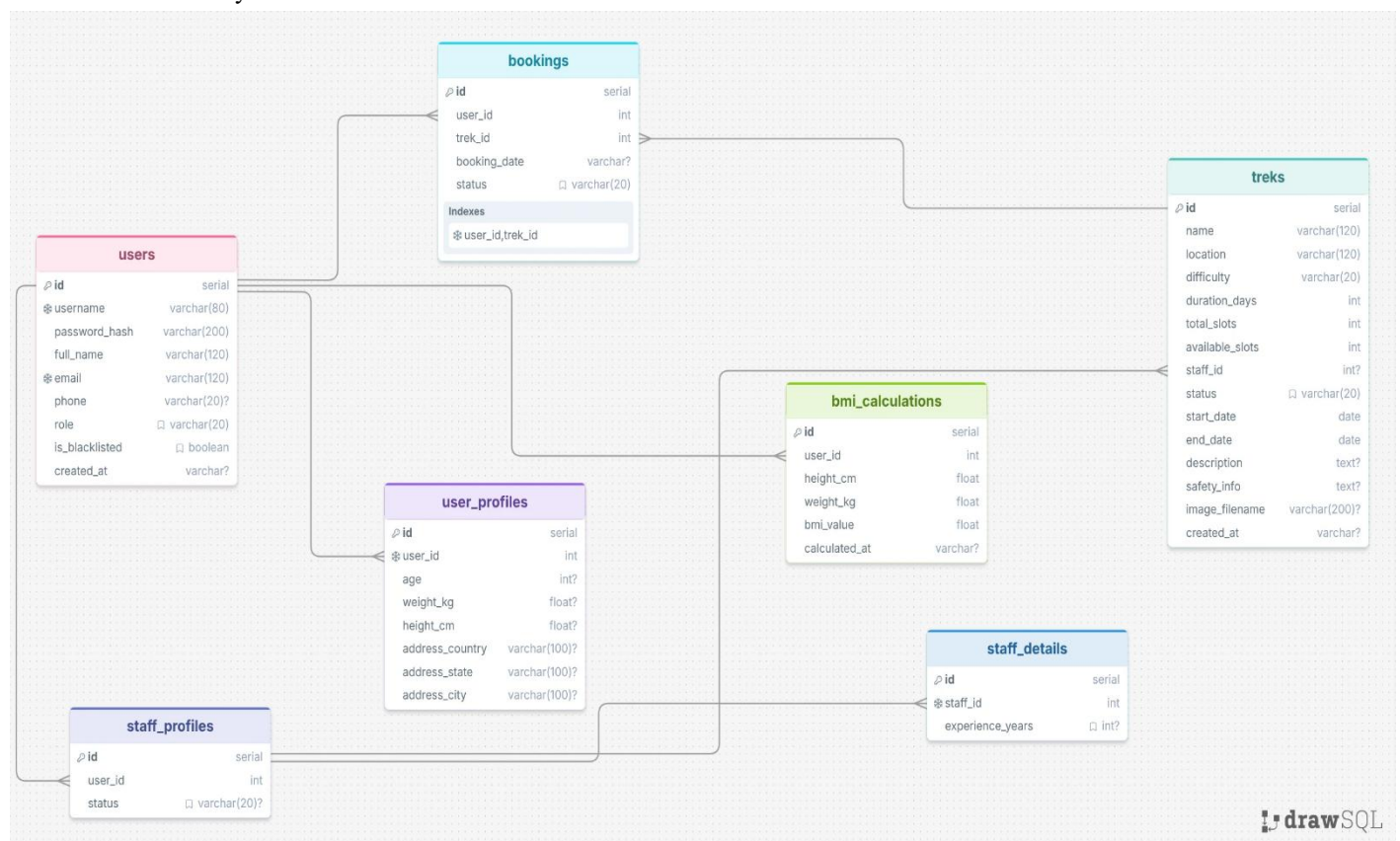
5. Database Schema / ER Diagram

Tables:

- User — stores account credentials and profile info (id, username, password_hash, full_name, email, phone, role, is_blacklisted)
- UserProfile — extended trekker details (age, weight, height, address)
- StaffProfile — links a User to their staff role, tracks approval status (pending / active / rejected) and assigned treks
- StaffDetails — additional staff info such as years of experience
- Trek — trek listings (name, location, difficulty, duration, total/available slots, status, dates, description, safety info, assigned staff)
- Booking — links a User to a Trek with a status (Booked / Cancelled) and a uniqueness constraint preventing duplicate bookings
- BMICalculation — stores BMI readings calculated for a user over time

Relationships:

- One-to-One → User ↔ UserProfile
- One-to-One → User → StaffProfile (for staff accounts)
- One-to-One → StaffProfile → StaffDetails
- One-to-Many → StaffProfile → Trek (a staff member can be assigned multiple treks)
- One-to-Many → User → Booking
- One-to-Many → Trek → Booking
- One-to-Many → User → BMICalculation



6. Application Routes (by Role)

Auth

Route	Method	Description
/	GET	Public home page
/login	GET, POST	Login for all three roles
/register	GET, POST	Trekker (user) self-registration
/register-staff	GET, POST	Trek staff self-registration (requires admin approval)
/logout	GET	Ends the current session

Admin

Route	Method	Description
/admin/dashboard	GET	Overview stats and pending approvals
/admin/treks	GET	List all treks
/admin/treks/add	GET, POST	Create a new trek
/admin/treks/<id>/edit	GET, POST	Edit trek details
/admin/treks/<id>/assign-staff	POST	Assign a staff member to a trek
/admin/staff	GET	List staff and pending staff requests
/admin/staff/<id>/approve	POST	Approve a pending staff registration
/admin/staff/<id>/reject	POST	Reject a pending staff registration
/admin/users	GET	List all trekkers
/admin/blacklist/<id>	POST	Toggle blacklist status for a user
/admin/bookings	GET	List all bookings
/admin/search	GET	Search users, staff, treks, bookings by display ID
/admin/analytics	GET	Aggregate booking/trek analytics

Staff

Route	Method	Description
/staff/dashboard	GET	Staff overview
/staff/assigned-treks	GET	List treks assigned to this staff member
/staff/treks/<id>/manage	GET, POST	Update trek status/details (only if assigned)
/staff/treks/<id>/participants	GET	View trekkers booked on this trek
/staff/profile/edit	GET, POST	Edit own staff profile

User (Trekker)

Route	Method	Description
/user/dashboard	GET	Trekker overview
/user/treks	GET	Browse available treks
/user/treks/<id>/book	GET, POST	Book a trek (blocked if full/not open)

Route	Method	Description
/user/bookings/<id>/cancel	POST	Cancel a booking (slot returned)
/user/history	GET	Past trek history
/user/profile/edit	GET, POST	Edit own profile

7. Architecture and Features

Architecture Overview:

- app.py — application factory; registers blueprints and seeds the default admin account
- backend/models.py — all SQLAlchemy database models
- backend/routes.py — all route logic, organized into four sections: Auth, Admin, Staff, User
- templates/ — Jinja2 HTML templates, split into auth/, admin/, staff/, and user/ folders, sharing base.html and layout_dashboard.html
- static/css — custom styling on top of Bootstrap; static/upload — uploaded trek photos
- tests/ — automated tests, e.g. for display-ID generation (U001, S001, T001, B001 style IDs)

Implemented Features:

- Role-based authentication for Admin, Trek Staff, and Trekker with a route-level access-control decorator
- Self-registration with admin-approval workflow for trek staff (pending / active / rejected states)
- Trek creation, editing, and staff assignment by Admin
- Slot-aware trek booking for trekkers — booking is blocked once a trek is full or not Open
- Booking cancellation that preserves history (status changed to Cancelled, slot returned) rather than deleting records
- Staff-side participant management, restricted to the staff member assigned to that trek
- Admin search across users, staff, treks, and bookings by human-readable display IDs (e.g. U001, T003)
- Admin analytics dashboard summarizing bookings and trek activity
- Blacklisting of users/staff, enforced at login
- Secure password hashing via Werkzeug

Additional Features:

- BMI calculation and tracking for trekkers
- Automated tests for display-ID formatting logic

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1bG6CvqDSCvaXQNOBLrNSv4DLHHoiBa1u/view?usp=sharing>